CSED233 · 데이터구조 · 포스텍

정리본 8. Sorting

단순 정렬 vs 고급 정렬 · 각 알고리즘의 동작 · 언제 무엇을 고를까.

1. 정렬이란 + 알고리즘 지도

Sorting(정렬): (key, info) 쌍으로 이루어진 원소 컬렉션을 key 기준으로 재배열하는 것.

단순 정렬 · $O(n^2)$

- Bubble Sort(거품 정렬)
- Selection Sort(선택 정렬)
- Insertion Sort(삽입 정렬)

구현이 간단, 작은 데이터용.

고급 정렬 · $O(n \log n)$

- Merge Sort(병합 정렬)
- Quick Sort(퀵 정렬)
- Heap Sort(힙 정렬)

구현은 복잡하지만 실무 표준.

특수 정렬

- Counting Sort(계수 정렬)
- Radix Sort(기수 정렬)

특정 데이터 타입에서만 가능.

2. 단순 정렬 ($O(n^2)$)

2.1 Bubble Sort(버블 정렬)

인접한 두 원소를 비교·교환하며 가장 큰 값을 맨 뒤로 떠오르게 한다. 한 Pass마다 맨 뒤 한 칸이 확정.

예제 · [5, 3, 8, 1, 2]

```
Pass 1: (5,3)→교환 [3,5,8,1,2] · (5,8)→유지 · (8,1)→교환 · (8,2)→교환
      [3, 5, 1, 2, |8|]  ← 8 확정
Pass 2: (3,5)→유지 · (5,1)→교환 · (5,2)→교환
      [3, 1, 2, |5, 8|]  ← 5 확정
Pass 3: (3,1)→교환 · (3,2)→교환
      [1, 2, |3, 5, 8|]  완성
```

시간: 최선 $O(n)$ (교환이 없으면 조기 종료하는 최적화 적용 시), 평균/최악 $O(n^2)$ · 공간 $O(1)$ · Stable ✓

2.2 Selection Sort(선택 정렬)

정렬 안 된 부분에서 최솟값을 찾아 맨 앞과 교환한다. 한 Pass마다 맨 앞 한 칸이 확정.

예제 · [5, 3, 8, 1, 2]

```
Pass 1: 최솟값 1 → 맨 앞과 swap → [1, 3, 8, 5, 2]
Pass 2: 최솟값 2 → 두 번째와 swap → [1, 2, 8, 5, 3]
Pass 3~: [1, 2, 3, 5, 8] 완성
```

시간: 항상 $O(n^2)$ · 공간 $O(1)$ · **Unstable ✗** · swap은 최대 n−1회(Bubble보다 적음)

2.3 Insertion Sort(삽입 정렬)

정렬된 앞부분을 유지하며, 다음 원소를 알맞은 위치에 삽입한다.

예제 · [5, 3, 8, 1, 2]

```
[5 | 3, 8, 1, 2]   3 삽입 → [3, 5 | 8, 1, 2]
[3, 5 | 8, 1, 2]   8 삽입 → [3, 5, 8 | 1, 2]
…                  → [1, 2, 3, 5, 8] 완성
```

거의 정렬된 데이터

Insertion Sort는 **거의 정렬된 입력에서** $O(n)$에 가깝게 동작한다 — 각 원소가 거의 이동하지 않기 때문. 실무 라이브러리가 작은 구간에서 삽입 정렬로 전환하는 이유.

시간: 최선 $O(n)$, 평균/최악 $O(n^2)$ · 공간 $O(1)$ · Stable ✓

3. 고급 정렬 ($O(n \log n)$)

3.1 Merge Sort(병합 정렬) — 분할 정복

① **분할**: 배열을 반씩 계속 쪼갠다 — 깊이 $O(\log n)$.

② **병합**: 두 정렬된 조각을 합치며 정렬 — 각 층 $O(n)$.

예제 · [5, 3, 8, 1, 2, 9]

```
분할: [5,3,8] [1,2,9] → [5,3][8] [1,2][9] → [3][5] ⋯
병합: [3,5] + [8] → [3,5,8] · [1,2] + [9] → [1,2,9]
    [3,5,8] + [1,2,9] → [1, 2, 3, 5, 8, 9]
```

시간: **항상 $O(n \log n)$ — 최악에도 보장** · 공간 $O(n)$(임시 배열) · Stable ✓ · 실행시간 예측이 중요한 환경에 적합

Non-recursive Merge Sort (Bottom-up)

재귀 분할 없이, 배열을 이미 쪼개져 있다고 보고 **width = 1, 2, 4, ⋯** 단위로 이웃 조각을 병합해 올라간다. 각 단계 $O(n)$, 단계 수 $O(\log n)$ → 총 $O(n \log n)$. 재귀 호출 오버헤드와 stack overflow 위험이 없어 대용량에 유리.

예제 · [5, 3, 8, 1, 2, 9]

```
width 1: [5][3] [8][1] [2][9] → [3,5] [1,8] [2,9]
width 2: [3,5] [1,8] → [1,3,5,8] · [2,9]
width 4: [1,3,5,8] + [2,9] → [1, 2, 3, 5, 8, 9]
```

3.2 Quick Sort(퀵 정렬) — Pivot 기반 분할

① **Pivot 선택**(예: 맨 왼쪽 또는 중간값).

② **분할**: pivot 미만 | pivot | pivot 이상으로 나눈다 (같은 값은 오른쪽으로) — $O(n)$.

③ 양쪽을 **재귀** 정렬. pivot이 잘 갈라지면 깊이 $O(\log n)$.

예제 · [5, 3, 8, 1, 2], pivot = 5

```
<5: [3,1,2]  =5: [5]  >5: [8]
재귀: [3,1,2] → [1,2,3]
최종: [1, 2, 3, 5, 8]
```

시간: 평균 $O(n \log n)$, **최악 $O(n^2)$(pivot 선택 실패 시)** · 공간 평균 $O(\log n)$, 최악 O(n)(재귀 스택) · Unstable ✗ · **실제로는 가장 빠름(캐시 효율)**

In-place Partitioning (Two-Pointer Scan)

양 끝 포인터 L, R이 서로를 향해 이동하며 교환. L은 pivot 이상인 값, R은 pivot 미만인 값을 찾아 멈춘다.

예제 · [17, 20, 17, 33, 15, 2, 21, 14], pivot = 17

```
L: 20(≥17) 발견, R: 14(<17) 발견 → swap
 [17, 14, 17, 33, 15, 2, 21, 20]
L: 17 발견, R: 2 발견 (21은 ≥17이라 스킵) → swap
 [17, 14, 2, 33, 15, 17, 21, 20]
L: 33 발견, R: 15 발견 → swap
 [17, 14, 2, 15, 33, 17, 21, 20]
L과 R이 엇갈림 → pivot을 R 위치(15)와 교환
 [15, 14, 2] | 17 | [33, 17, 21, 20]
→ 좌우를 각각 재귀 → 최종 [2, 14, 15, 17, 17, 20, 21, 33]
```

> **엇갈린 뒤의 마지막 교환**
>
> 스캔이 끝나면 pivot은 **R이 가리키는 위치**와 교환한다(L이 아니라 R). 그래야 pivot 왼쪽이 모두 pivot 이하가 된다.

3.3 Heap Sort(힙 정렬)

배열을 Max-Heap으로 구성(정리본 7의 Heapify, $O(n)$) → 루트(최댓값)를 반복 추출. 시간 **항상** $O(n \log n)$, 공간 $O(1)$(in-place), Unstable ✗. 항상 일정한 성능이 강점.

4. 정렬 비교표

알고리즘	최선	평균	최악	공간	안정성
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗

5. 어떤 정렬을 고를까

데이터 크기별

- 작은 데이터 (예: n ≤ 50) → **Insertion**: 단순, 오버헤드 최소
- 중간 데이터 (예: 50 < n ≤ 1M) → **Quick**: 평균 최고 속도
- 큰 데이터 (예: n > 1M) → **Merge**: 최악 성능 보장

데이터 특성별

- 거의 정렬됨 → **Insertion** ($O(n)$에 근접)
- 안정성 필수(다중 키 정렬) → **Merge / Bubble / Insertion**
- 메모리 제약(임베디드) → **Quick / Heap**
- 최악 성능 보장 필요(실시간, 금융) → **Merge / Heap**

6. Quiz

Q1. 거의 정렬된 배열에서 가장 빠른 정렬은?

> 힌트

최선 시간복잡도가 $O(n)$인 알고리즘을 찾아보자.

> 정답

개념은 힌트 참고.

정답: Insertion Sort — 최선 $O(n)$. 다른 고급 정렬들은 입력과 무관하게 $O(n \log n)$.

Q2. Stable sort가 필요한 이유는?

> 힌트

키가 같은 원소들의 상대 순서가 유지되는가.

> 정답

개념은 힌트 참고.

정답: 다중 키 정렬(예: 급여 정렬 후 직급 정렬)에서 이전 정렬 순서를 보존하기 위해. Stable 정렬을 뒤 키부터 적용하면 이전 키 순서가 유지된다.

Q3. Merge Sort가 Quick Sort보다 느린 편인데 왜 쓸까?

> 힌트

비교표의 "최악" 열: Quick은 $O(n^2)$, Merge는 $O(n \log n)$.

> 정답

개념은 힌트 참고.

정답: Merge는 최악에도 $O(n \log n)$이 보장된다. Quick은 최악 $O(n^2)$이라 예측 불가라 실시간 시스템에 부적합.